

# Module 5: Characters and Strings

---

Sarah Foss

# Objectives

---

Understand the **char** data type and its relation to Unicode, ASCII, and numeric representation.

---

Learn how to use **escape sequences** and manage **whitespace characters**.

---

Explore character comparisons, and methods from the Character class like **isDigit()**, **toLowerCase()**, and **toUpperCase()**.

---

Understand String operations, including concatenation, **substring()**, comparison with **equals()** and **compareTo()**, and formatted output with **printf()**.

# char

---

A **char** is a primitive data type that represents a single character, such as a letter, digit, or symbol.

A **char** literal is enclosed in matching single quotes (').

Examples:

```
char letter = 'a' ;
```

```
char number = '1' ;
```

# Numeric Representation

---

Every **char** also has a numeric representation

We need numeric representation for **chars** because computers can only process numbers, not letters or symbols directly.

By assigning a unique number to each character, we give computers a way to store, manipulate, and compare text efficiently.

# ASCII Table

| Dec | Hex | Oct | Char | Dec | Hex | Oct | Char    | Dec | Hex | Oct | Char | Dec | Hex | Oct | Char |
|-----|-----|-----|------|-----|-----|-----|---------|-----|-----|-----|------|-----|-----|-----|------|
| 0   | 0   | 0   |      | 32  | 20  | 40  | [space] | 64  | 40  | 100 | @    | 96  | 60  | 140 | `    |
| 1   | 1   | 1   |      | 33  | 21  | 41  | !       | 65  | 41  | 101 | A    | 97  | 61  | 141 | a    |
| 2   | 2   | 2   |      | 34  | 22  | 42  | "       | 66  | 42  | 102 | B    | 98  | 62  | 142 | b    |
| 3   | 3   | 3   |      | 35  | 23  | 43  | #       | 67  | 43  | 103 | C    | 99  | 63  | 143 | c    |
| 4   | 4   | 4   |      | 36  | 24  | 44  | \$      | 68  | 44  | 104 | D    | 100 | 64  | 144 | d    |
| 5   | 5   | 5   |      | 37  | 25  | 45  | %       | 69  | 45  | 105 | E    | 101 | 65  | 145 | e    |
| 6   | 6   | 6   |      | 38  | 26  | 46  | &       | 70  | 46  | 106 | F    | 102 | 66  | 146 | f    |
| 7   | 7   | 7   |      | 39  | 27  | 47  | '       | 71  | 47  | 107 | G    | 103 | 67  | 147 | g    |
| 8   | 8   | 10  |      | 40  | 28  | 50  | (       | 72  | 48  | 110 | H    | 104 | 68  | 150 | h    |
| 9   | 9   | 11  |      | 41  | 29  | 51  | )       | 73  | 49  | 111 | I    | 105 | 69  | 151 | i    |
| 10  | A   | 12  |      | 42  | 2A  | 52  | *       | 74  | 4A  | 112 | J    | 106 | 6A  | 152 | j    |
| 11  | B   | 13  |      | 43  | 2B  | 53  | +       | 75  | 4B  | 113 | K    | 107 | 6B  | 153 | k    |
| 12  | C   | 14  |      | 44  | 2C  | 54  | ,       | 76  | 4C  | 114 | L    | 108 | 6C  | 154 | l    |
| 13  | D   | 15  |      | 45  | 2D  | 55  | -       | 77  | 4D  | 115 | M    | 109 | 6D  | 155 | m    |
| 14  | E   | 16  |      | 46  | 2E  | 56  | .       | 78  | 4E  | 116 | N    | 110 | 6E  | 156 | n    |
| 15  | F   | 17  |      | 47  | 2F  | 57  | /       | 79  | 4F  | 117 | O    | 111 | 6F  | 157 | o    |
| 16  | 10  | 20  |      | 48  | 30  | 60  | 0       | 80  | 50  | 120 | P    | 112 | 70  | 160 | p    |
| 17  | 11  | 21  |      | 49  | 31  | 61  | 1       | 81  | 51  | 121 | Q    | 113 | 71  | 161 | q    |
| 18  | 12  | 22  |      | 50  | 32  | 62  | 2       | 82  | 52  | 122 | R    | 114 | 72  | 162 | r    |
| 19  | 13  | 23  |      | 51  | 33  | 63  | 3       | 83  | 53  | 123 | S    | 115 | 73  | 163 | s    |
| 20  | 14  | 24  |      | 52  | 34  | 64  | 4       | 84  | 54  | 124 | T    | 116 | 74  | 164 | t    |
| 21  | 15  | 25  |      | 53  | 35  | 65  | 5       | 85  | 55  | 125 | U    | 117 | 75  | 165 | u    |
| 22  | 16  | 26  |      | 54  | 36  | 66  | 6       | 86  | 56  | 126 | V    | 118 | 76  | 166 | v    |
| 23  | 17  | 27  |      | 55  | 37  | 67  | 7       | 87  | 57  | 127 | W    | 119 | 77  | 167 | w    |
| 24  | 18  | 30  |      | 56  | 38  | 70  | 8       | 88  | 58  | 130 | X    | 120 | 78  | 170 | x    |
| 25  | 19  | 31  |      | 57  | 39  | 71  | 9       | 89  | 59  | 131 | Y    | 121 | 79  | 171 | y    |
| 26  | 1A  | 32  |      | 58  | 3A  | 72  | :       | 90  | 5A  | 132 | Z    | 122 | 7A  | 172 | z    |
| 27  | 1B  | 33  |      | 59  | 3B  | 73  | ;       | 91  | 5B  | 133 | [    | 123 | 7B  | 173 | {    |
| 28  | 1C  | 34  |      | 60  | 3C  | 74  | <       | 92  | 5C  | 134 | \    | 124 | 7C  | 174 |      |
| 29  | 1D  | 35  |      | 61  | 3D  | 75  | =       | 93  | 5D  | 135 | ]    | 125 | 7D  | 175 | }    |
| 30  | 1E  | 36  |      | 62  | 3E  | 76  | >       | 94  | 5E  | 136 | ^    | 126 | 7E  | 176 | ~    |
| 31  | 1F  | 37  |      | 63  | 3F  | 77  | ?       | 95  | 5F  | 137 | _    | 127 | 7F  | 177 |      |

American Standard Code for Information Interchange

# Decimal Values for Common Characters

| Characters | Decimal Value |
|------------|---------------|
| '0' to '9' | 48 to 57      |
| 'A' to 'Z' | 65 to 90      |
| 'a' to 'z' | 97 to 122     |

```
char let = (char)97;  // 'a'  
char num = (char)49;  // '1'
```

# Beyond ASCII - Unicode

---

While ASCII works well for English text, many languages, such as Chinese, need a much larger set of symbols to cover their alphabets.

The Unicode standard uses 16-bit (2-byte) patterns to represent characters from all major languages.

The first 256 characters in Unicode are identical to ASCII

$2^{16} = 65536$  Unicode symbols

# Unicode Values

You can represent any Unicode character using a `\u` followed by its four-digit hexadecimal (base 16) value.

This system allows you to include any character from the vast Unicode set in your code, even ones that can't be easily typed from a keyboard. It ensures that characters from different languages can all be represented consistently in Java

| Characters | Unicode Decimal Value | Unicode Hexadecimal Value |
|------------|-----------------------|---------------------------|
| '0' to '9' | 48 to 57              | \u0030 to \u0039          |
| 'A' to 'Z' | 65 to 90              | \u0041 to \u005A          |
| 'a' to 'z' | 97 to 122             | \u0061 to \u007A          |

```
char uLet = '\u0061' ;  
char uNum = '\u0031' ;
```



# Escape Character

---

An escape character is a special character used to represent a character that can't be typed directly or is reserved for specific purposes in code.

In Java, the escape character is the backslash (\).

When combined with certain characters, it forms an escape sequence, which tells the compiler to interpret it as a special character.

# Common Escape Sequences

---

Some common escape sequences, along with their decimal and Unicode values:

| Escape Sequence | Description  | Unicode Decimal Value | Unicode Hexadecimal Value |
|-----------------|--------------|-----------------------|---------------------------|
| <code>\n</code> | Newline      | 10                    | <code>\u000A</code>       |
| <code>\t</code> | Tab          | 9                     | <code>\u0009</code>       |
| <code>\'</code> | Single Quote | 39                    | <code>\u0027</code>       |
| <code>\"</code> | Double Quote | 34                    | <code>\u0022</code>       |
| <code>\\</code> | Backslash    | 92                    | <code>\u005C</code>       |

# Character Arithmetic

---

Since each char has an underlying numeric value, you can perform arithmetic operations on char values in a similar way you would with integers.

## Incrementing/Decrementing

```
char letter = 'A';  
letter++;  
System.out.println(letter); // Output: B
```

# Character Arithmetic

---

It's important to follow Java's type conversion rules carefully when performing these operations to avoid unexpected results.

## **Adding/Subtracting a Number:**

```
char letter = 'A';    // 'A' has the Unicode value 65
char newLetter = (char) (letter + 2); // 'C' (65 + 2 = 67)
System.out.println(newLetter); // Output: C
```

## **Character Distance Calculation:**

```
char letterA = 'A'; // Unicode 65
char letterE = 'E'; // Unicode 69
int difference = letterE - letterA; // 69 - 65 = 4
System.out.println(difference); // Output: 4
```

# Comparing Characters

---

Two characters can be compared using relational operators:

```
'a' < 'b'
```

```
//true because the Unicode for 'a' (97) is less than the  
Unicode for 'b' (98).
```

```
'a' < 'A'
```

```
//false because the Unicode for 'a' (97) is greater than the  
Unicode for 'A' (65).
```

```
'1' < '8'
```

```
//true because the Unicode for '1' (49) is less than the  
Unicode for '8' (56).
```

# Using Characters in Loops

---

You can use char in loops by incrementing or decrementing to iterate over characters.

```
for (char letter = 'A'; letter <= 'Z'; letter++) {  
    System.out.print(letter + " ");  
    // Output: A B C ... Z  
}
```

# Character Methods

The **Character** class in Java provides several useful methods for working with **char** values.

These methods allow you to check properties of characters, convert cases, and perform other character-related operations.

| Method                                | Description                                                                            |
|---------------------------------------|----------------------------------------------------------------------------------------|
| <code>isDigit(char ch)</code>         | Returns true if the character is a digit (0-9).                                        |
| <code>isLetter(char ch)</code>        | Returns true if the character is a letter (A-Z or a-z).                                |
| <code>isLetterOrDigit(char ch)</code> | Returns true if the character is either a letter or a digit.                           |
| <code>isLowerCase(char ch)</code>     | Returns true if the character is a lowercase letter (a-z).                             |
| <code>isUpperCase(char ch)</code>     | Returns true if the character is an uppercase letter (A-Z).                            |
| <code>toLowerCase(char ch)</code>     | Converts the character to lowercase if it's an uppercase letter.                       |
| <code>toUpperCase(char ch)</code>     | Converts the character to uppercase if it's a lowercase letter.                        |
| <code>isWhitespace(char ch)</code>    | Returns true if the character is a whitespace character (like space, tab, or newline). |

# Explanation:

**isDigit(char ch):** Checks if a character is a digit between '0' and '9'.

```
Character.isDigit('5'); // true
```

**isLetter(char ch):** Checks if a character is a letter, covering both uppercase and lowercase.

```
Character.isLetter('A'); // true
```

```
Character.isLetter('7'); // false
```

**isLowerCase(char ch) and isUpperCase(char ch):** These methods determine if a character is in lowercase or uppercase.

```
Character.isLowerCase('a'); // true
```

```
Character.isUpperCase('A'); // true
```

**toLowerCase(char ch) and toUpperCase(char ch):** Converts characters to their lowercase or uppercase equivalents.

```
Character.toLowerCase('A'); // 'a'
```

```
Character.toUpperCase('a'); // 'A'
```

**isWhitespace(char ch):** Identifies if a character is a whitespace (spaces, tabs, or newlines).

```
Character.isWhitespace(' '); // true
```

```
Character.isWhitespace('\n'); // true
```



# Strings

---

A **string** is a sequence of characters.

A **string** literal is enclosed in matching double quotes (").

Example:

```
String personName = "Agatha Harkness";  
System.out.println("My name is " + personName);
```

# Strings are Reference Types

---

A **String** in Java is actually a predefined class in the Java library, similar to the **Scanner** class.

Unlike primitive types like **int** or **char**, the **String** type is a **reference type**, meaning they have methods and properties that can be used to manipulate and work with text.

Any class can be used to create a reference type, allowing you to create objects and use them in your programs.

# String Literal Rules

---

- **String** literals must be enclosed in double quotes (").
- **Strings** can be as long as 2,147,483,647 characters.
- A **string** can have zero characters, represented by "", which is known as the **empty string**.
- When displayed, quotation marks are removed from **string** literals.
- Certain special characters in **strings** must be represented with escape sequences.
- **String** literals cannot span multiple lines directly. To include line breaks, you must use escape sequences (`\n`) or text blocks:

```
String multiLine = "This is line 1\nThis is line 2";
```

```
String textBlock = """
```

```
    This is line 1
```

```
    This is line 2
```

```
    """;
```

# Concatenation

---

String concatenation is the process of joining two or more strings together to form a single string.

This can be done using the + operator:

```
String firstName = "Wade";  
String lastName = "Wilson";  
String fullName = firstName + " " + lastName;
```

# Concatenation

---

Java uses the + operator for both addition and string concatenation.

If one of the values is not a string (like an integer), Java automatically converts it to a string and then combines it with the other string.

If both values are numbers, the + operator performs addition.

Example:

```
int x = 10;  
int y = 20;  
int z = x + y;  
// z will be 30 (an  
integer)
```

```
String x = "10";  
String y = "20";  
String z = x + y;  
// z will be 1020 (a  
String)
```

```
String x = "10";  
int y = 20;  
String z = x + y;  
// z will be 1020 (a  
String)
```

# Reading Strings

---

The Scanner class provides different methods to read strings, such as:

**next ()** : Reads a single word (text up to the first space or newline).

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter three words separated by spaces: ");  
String s1 = input.next();  
String s2 = input.next();  
String s3 = input.next();  
System.out.println("s1 is " + s1);  
System.out.println("s2 is " + s2);  
System.out.println("s3 is " + s3);
```

# Reading Strings

---

The Scanner class provides different methods to read strings, such as:

**`nextLine()`** : Reads the entire line of text input from the console, including spaces, until the user presses enter.

```
Scanner input = new Scanner(System.in);  
System.out.println("Enter a line: ");  
String s = input.nextLine();  
System.out.println("The line entered is " + s);
```

## Note:

If you mix `nextLine()` with other `next()` methods, you might need to use `nextLine()` to consume the newline character left by `next()`.

# String Methods

---

In Java, a String is an object that comes with various methods to perform operations on the string.

These methods can only be used with a specific string object, which is why they are referred to as **instance methods**.

Syntax:

```
referenceVariable.methodName(arguments)
```

Example:

```
String greeting = "Hello, World!";  
int length = greeting.length(); // Returns the length of the string  
System.out.println("The length of the string is: " + length);
```



# String Methods

---

The String class provides a wide range of methods for string manipulation.

| Method                                               | Description                                                                                                                                |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>length()</code>                                | Returns the number of characters in the string.                                                                                            |
| <code>trim()</code>                                  | Removes leading and trailing whitespace.                                                                                                   |
| <code>toLowerCase()</code>                           | Converts all characters in the string to lowercase.                                                                                        |
| <code>toUpperCase()</code>                           | Converts all characters in the string to uppercase.                                                                                        |
| <code>replace(char oldChar, char newChar)</code>     | Replaces all occurrences of a character with another character.                                                                            |
| <code>charAt(int index)</code>                       | Returns the character at the specified index.                                                                                              |
| <code>equals(String s1)</code>                       | Compares two strings for exact equality.                                                                                                   |
| <code>compareTo(String s1)</code>                    | Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.. |
| <code>substring(int beginIndex)</code>               | Returns a substring starting from the specified index.                                                                                     |
| <code>substring(int beginIndex, int endIndex)</code> | Returns a substring between the specified start and end indexes.                                                                           |
| <code>indexOf(String str)</code>                     | Returns the index of the first occurrence of the specified string.                                                                         |
| <code>indexOf(int ch, int fromIndex)</code>          | Finds the index of the specified character (ch), starting the search from the given index (fromIndex).                                     |
| <code>split(String regex)</code>                     | Splits the string based on a regular expression and returns an array.                                                                      |

**trim():** removes any leading and trailing whitespace (spaces, tabs, or line breaks) from a string. It doesn't affect the whitespace inside the string.

```
messyString = "    I'm Mary Poppins, Y'all!    ";  
String cleanedString = messyString.trim();  
System.out.println(cleanedString); // Output: I'm Mary Poppins, Y'all!
```

**toUpperCase():** converts all characters in a string to uppercase.

```
String lowercase = "i am groot";  
String uppercase = lowercase.toUpperCase();  
System.out.println(uppercase); // Output: I AM GROOT
```

**toLowerCase():** converts all characters in a string to lowercase.

```
mixedCase = "I'm mARveL JeSUs!";  
System.out.println(mixedCase.toLowerCase()); // Output: i'm marvel jesus
```

**replace():** replaces all occurrences of a character in a string with another character.

```
String text = "Peter Parker";  
String newText = text.replace('a', 'o'); // Replaces all 'a' with 'o'  
System.out.println(newText); // Output: Perter Porker
```

# Getting Characters from a String

---

The `charAt()` method is used to get the character at a specific index within a string.  
The index of the first character in a string is 0.

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|-------|---|---|---|---|---|---|---|---|---|---|----|----|
|       | Y | o | u |   | g | o | t |   | t | h | i  | s  |

`message.charAt(0)` is Y

`message.length()` is 12

`message.charAt(11)` is s

```
String message = "You got this";  
System.out.println("The first character in message is " +  
message.charAt(0));  
System.out.println("The last character in message is " +  
message.charAt(message.length() - 1));
```

# Comparing Strings

---

**Strings are compared using the equals() method, not the == operator.**

The == operator checks if two strings refer to the same memory location, while equals() checks if the content of the strings is the same.

```
String str1 = "hello";
String str2 = new String("hello");
if (str1.equals(str2)) {
    System.out.println("Strings are equal"); // True
}
if (str1 == str2) {
    System.out.println("Same memory location"); // False
}
```

# Comparing Strings

The **compareTo()** method in Java compares two strings in alphabetical (lexicographical) order based on their Unicode values.

It checks each character from left to right until it finds a difference.

If one string is shorter but matches the beginning of the other, their lengths decide the result.

## Syntax:

```
string1.compareTo(string2)
```

## Returns:

- A **positive number** if string1 comes after string2.
- A **negative number** if string1 comes before string2.
- **0** if the strings are **equal**.

```
String str1 = "apple", str2 = "banana", str3 = "apple";  
System.out.println(str1.compareTo(str2)); // Output: -1  
System.out.println(str1.compareTo(str3)); // Output: 0
```

# Substrings

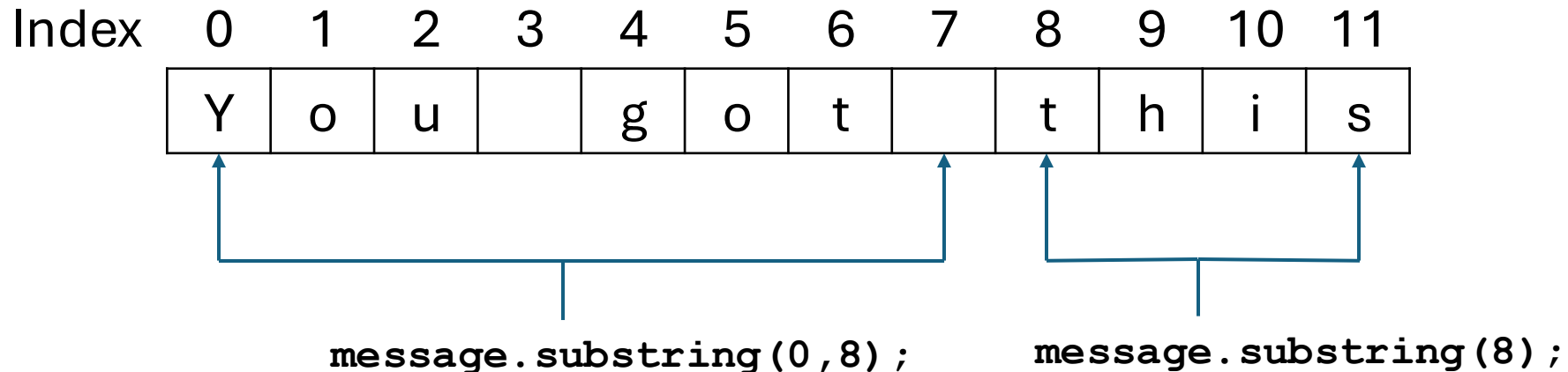
---

The `substring()` method in Java is used to extract a portion (or substring) of a string.

There are two main versions of `substring()`:

**`substring(int beginIndex)`** : Returns a substring starting from the specified index (inclusive).

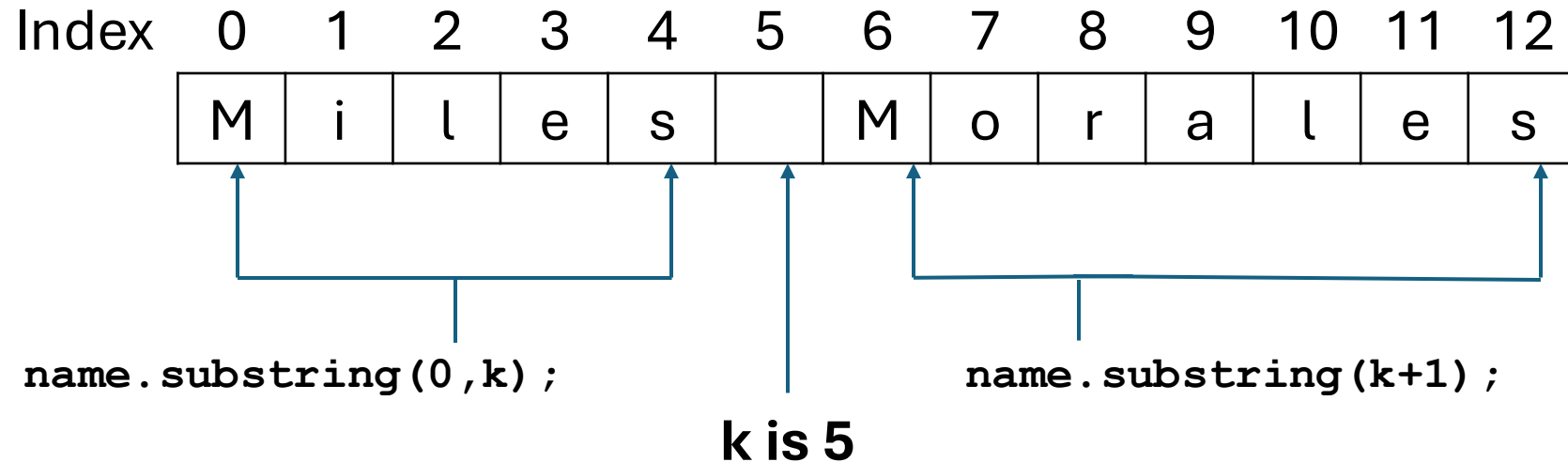
**`substring(int beginIndex, int endIndex)`** : Returns a substring between the specified start(inclusive) and end(exclusive) indexes (it does not include the character at `endIndex`)



# Finding a Character or Substring in a String

---

```
String name = "Miles Morales";  
int k = name.indexOf(' ');  
String firstName = name.substring(0, k);  
String lastName = name.substring(k + 1);
```



# Iterating Through Strings

---

Since strings are essentially sequences of characters, you can use loops to access each character individually.

```
String text = "We know each other! He's a friend from work!";  
for (int i = 0; i < text.length(); i++) {  
    char c = text.charAt(i);  
    System.out.println("Character at index " + i + ": " + c);  
}
```



# Converting Between Strings and Numbers

---

## Converting Strings to Numbers:

`Integer.parseInt(String s)`: Converts a **String** to an **int**.

```
int intValue = Integer.parseInt(intString);
```

`Double.parseDouble(String s)`: Converts a **String** to a **double**.

```
double doubleValue = Double.parseDouble(doubleString);
```

*Note: If the string cannot be parsed as a number, it will cause a runtime error*

## Converting Numbers to Strings:

Concatenating an empty string to a number converts it to a **String**.

```
String str = 39 + "";
```

# Formatting Output – printf()

---

`printf()` is used to print formatted output to the console.

**Syntax:**

```
System.out.printf(format, items);
```

Where format is a string that may consist of substrings and format specifiers.

```
int age = 25;
```

```
double height = 5.9;
```

```
String name = "Alice";
```

```
System.out.printf("Name: %s, Age: %d, Height: %.1f feet\n", name, age, height);
```

The diagram illustrates the mapping of variables to format specifiers in the `printf` statement. An orange line connects the `name` variable to the `%s` specifier. A purple line connects the `age` variable to the `%d` specifier. A green line connects the `height` variable to the `%.1f` specifier.

# Format Specifiers

---

- You can format numbers, characters, Booleans, or strings.
- A format specifier tells how to display an item and starts with %.
- To print a literal %, use %% in the format string.
- Items must follow the format specifiers in the same order, type, and quantity.
- You can set the field width and precision within a format specifier.
- By default, the output is right-justified. To make it left-justified, add a minus sign (-) in the format specifier.

## Common Specifiers:

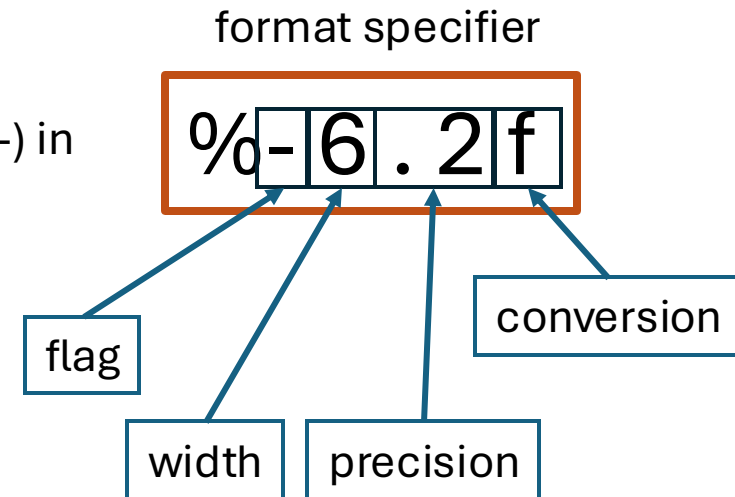
%d: Integer values

%f: Floating-point numbers

%s: Strings

%b: Boolean value

%c: character



# Summary

---

- The **char** data type represents a single character, such as a letter, digit, or symbol.
- A **char** is based on the Unicode standard, which expands the ASCII set.
- Each character has a numeric representation corresponding to its Unicode value.
- **Escape sequences** begin with a backslash (\) followed by a character or number.
- **Whitespace characters** include spaces, tabs, and line breaks.
- Characters are compared using their **Unicode values** with relational operators.
- The **Character** class includes methods like **isDigit()**, **isLetter()**, **isLowerCase()**, **isUpperCase()**, **toLowerCase()**, and **toUpperCase()**.

# Summary

---

- A **String** literal is enclosed in double quotes, while a **char** uses single quotes.
- Strings have **instance methods** like **length()**, **charAt()**, and **indexOf()**.
- Use the + operator to combine strings, and **substring()** to extract parts.
- **equals()** checks if strings are equal, while **compareTo()** compares their order.
- **System.out.printf()** is used for formatted output.